Deployment Readiness

What must change before this system can run in production

Verdict

The application is **not deployable to production as it stands**, but it is close. The gap is not missing features — it is that several things which are perfectly reasonable on a developer's laptop become data-loss or outage conditions the moment the app runs in a container behind a load balancer. The most serious is that **every uploaded document is destroyed on each redeploy**, because the storage path is hardcoded inside the application directory.

Sixteen changes are set out below. Five of them, totalling about an hour of work, are what stand between the current state and a system that can safely take real traffic on a single instance.

How to use this document

Every task states the exact file to change, why it matters, the change itself, and how to confirm it worked. An implementing agent should work them in the order given in **Recommended order** rather than top to bottom, and should not combine unrelated tasks into one commit.

> *A machine-readable copy of exactly these tasks, with the same code, is written alongside this report at **docs/DEPLOYMENT_TASKS.md**. Both files are generated from one source. An agent should prefer the Markdown file, because code indentation survives it exactly, whereas extracting text from a PDF can disturb whitespace.*

What is already production-grade

This is not a system that ignored operational concerns. The following are genuinely well built and should not be regressed while addressing the list that follows:

- Boot-time fail-fast checks for PostgreSQL, ClamAV and the KMS key provider — the process refuses to start degraded rather than failing on the first real request.
- Database migrations are serialised with a PostgreSQL advisory lock, so several instances booting at once cannot race each other into a broken schema.
- Security headers are thorough and deliberate: a strict Content-Security-Policy with no inline script, HSTS enabled outside development, nosniff, frame-deny and a sensible referrer policy.
- CORS is deny-by-default with an explicit allow-list, not a reflecting wildcard.
- A real health endpoint that separately reports PostgreSQL, ClamAV and KMS reachability, with caching, suitable for a load-balancer probe.
- Structured logging with a request id carried into responses, and one access-log line per request.
- Secrets are read from the environment and nothing is baked into the container image.

Recommended order

Stage 1 — before any production traffic

T1, T3, T6, T9, T11. Roughly one hour of work in total. These stop data loss, stop the firm-wide login lockout, stop deploys from truncating uploads, and stop an accidental seed from overwriting credentials.

Stage 2 — before the firm relies on it

T5, T7, T8, T10, T12, T15. Restores OCR in a locked-down network, authenticates the database connection, drops root in the container, makes CI meaningful, validates configuration at boot, and gives operations a runbook with a tested restore.

Stage 3 — before scaling beyond one instance

T2, T4. Shared object storage and a shared rate-limit counter. Not needed for a single-instance deployment, and a single instance is a perfectly respectable starting point for one firm.

Stage 4 — scheduled hardening

T13, T14, T16. Tightening and the session-cookie refactor, which deserves its own change and its own test pass.

Blockers — these prevent a safe deployment

T1. Make the document storage path configurable

BLOCKER | Any deployment. Causes permanent data loss. | Effort: 15 minutes

File(s): server/services/fileService.js (line 18)

STORAGE_ROOT is hardcoded to a path inside the application directory. In a container that is the image's own writable layer, which is discarded on every restart, redeploy and crash — so every document the firm has uploaded is destroyed. With more than one instance it also means a file uploaded through instance A cannot be downloaded from instance B, which returns a 404 for a document that plainly exists.

server/services/fileService.js — replace line 18

```
// Before:
const STORAGE_ROOT = path.join(__dirname, '..', '..', 'storage', 'uploads');

// After:
const STORAGE_ROOT = process.env.STORAGE_ROOT
  ? path.resolve(process.env.STORAGE_ROOT)
  : path.join(__dirname, '..', '..', 'storage', 'uploads');
```

Notes. resolveStoragePath()'s existing startsWith(STORAGE_ROOT) traversal guard keeps working, because path.resolve() returns an absolute path just as path.join(__dirname, ...) did. Then mount a persistent volume and set STORAGE_ROOT to its mount point, for example /var/lib/slf/uploads.

Verify. Upload a document, restart the container, and download the same document successfully. Then confirm the file exists on the mounted volume from outside the container.

T2. Add an object-storage adapter for multi-instance deployment

BLOCKER | Running more than one instance. Not required for a single instance with a volume (T1). | Effort: 3–4 hours

File(s): server/services/storage/providerSelector.js (new); server/services/storage/providers/localProvider.js (new); server/services/storage/providers/s3Provider.js (new); server/services/fileService.js (callers)

A mounted volume solves persistence for one instance but not sharing between instances. Most container platforms cannot attach one read-write volume to several replicas, so horizontal scaling requires object storage.

Follow the adapter pattern this codebase already uses four times

```
// Copy the exact shape of server/services/esignature/providerSelector.js:
const PROVIDERS = { local: localProvider, s3: s3Provider };

function getStorageProvider() {
  const name = (process.env.STORAGE_PROVIDER || 'local').toLowerCase();
  const provider = PROVIDERS[name];
  if (!provider) {
    throw new Error(`Unknown STORAGE_PROVIDER "${name}". Available: ${Object.keys(PROVIDERS).join(', ')}`);
  }
  return provider;
}

// Each provider implements exactly these four methods:
//   async write(relativePath, buffer)  -> void
//   async read(relativePath)           -> Buffer
//   async remove(relativePath)         -> void
//   async exists(relativePath)         -> boolean
```

Notes. Call sites to convert are all the fs.promises reads/writes in server/services/fileService.js, plus the fs.readFile in consultationService.bundleDocuments (which reads via resolveStoragePath). Keep storage_path_url in the database exactly as it is — it is already a relative, tenant-scoped key, which is exactly what an S3 object key wants to be. @aws-sdk/client-s3 is the natural dependency; @aws-sdk/client-kms is already in use, so the SDK family is already vendored.

Verify. Run two instances against the same database and STORAGE_PROVIDER=s3. Upload through instance A, download through instance B.

T3. Trust the reverse proxy so client IPs are real

BLOCKER | Any deployment behind a load balancer, ingress or CDN. | Effort: 10 minutes

File(s): server/server.js (after line 20)

Express is never told it sits behind a proxy, so req.ip is the proxy's address rather than the caller's. The login rate limiter therefore counts every user in the world into a single bucket of ten attempts per fifteen minutes — meaning roughly the eleventh login attempt by anyone locks out everyone, firm-wide, until the window expires. This is an outage, not a nuisance.

server/server.js — add immediately after const app = express();

```
// Behind a load balancer or ingress, req.ip is the proxy's address unless
// Express is told how many proxies to trust. express-rate-limit then buckets
// every caller together under that one address. Set this to the exact number
// of proxies in front of the app.
app.set('trust proxy', Number(process.env.TRUST_PROXY_HOPS || 0));
```

Notes. Set TRUST_PROXY_HOPS=1 behind a single load balancer, 2 behind a CDN in front of a load balancer. Do not use the value true: that trusts any X-Forwarded-For header the client sends, which makes the rate limit trivially bypassable by spoofing a new IP per request. Leave it 0 when the app is exposed directly.

Verify. Log req.ip behind the proxy and confirm it is the browser's address. Then make eleven failed logins from one machine (expect 429) and confirm a second machine can still log in.

T4. Move the rate limiter to a shared store

BLOCKER | Running more than one instance. | Effort: 1 hour

File(s): server/middleware/authRateLimit.js

express-rate-limit defaults to an in-memory store. Each instance therefore keeps its own counter, so N instances multiply the real limit by N, and every counter resets on deploy. Brute-force protection on the login and invite-acceptance endpoints is correspondingly weaker than intended.

server/middleware/authRateLimit.js — add a store

```
const { Pool } = require('pg');
const PostgresStore = require('@acpr/rate-limit-postgresql').default;

const authRateLimiter = rateLimit({
  windowMs,
  max,
  standardHeaders: true,
  legacyHeaders: false,
  // Shared across instances. Reuse the app's own pool config rather than a
  // second connection string.
  store: process.env.RATE_LIMIT_STORE === 'postgres'
    ? new PostgresStore({ session: { pool: require('../db/pool').pool }, prefix: 'auth_rl' })
    : undefined,
  handler(req, res) { /* unchanged */ }
});
```

Notes. Either a Postgres-backed store (no new infrastructure — the database is already there) or Redis via rate-limit-redis. Postgres is the lower-operational-cost choice here given no Redis exists yet. Leaving RATE_LIMIT_STORE unset keeps today's in-memory behaviour, which is correct for a single instance.

Verify. With two instances behind the load balancer, make ten failed logins spread across both. The eleventh must be refused regardless of which instance receives it.

T5. Vendor the Tesseract language data into the image

BLOCKER | Any deployment with restricted outbound internet. Degrades all others. | Effort: 45 minutes

File(s): server/services/ocr/providers/tesseractProvider.js (line 53); Dockerfile

createWorker('eng') is called with no language path, so tesseract.js downloads eng.traineddata (tens of megabytes) from a public CDN the first time anyone OCRs a scanned document, and caches it to the container's working directory. In a network that blocks outbound traffic this fails outright; everywhere else it re-downloads after every restart and writes into a filesystem layer that is about to be discarded. It also sends an outbound request at the exact moment a user is waiting on a scanned client document.

server/services/ocr/providers/tesseractProvider.js — pin the paths

```
const path = require('node:path');

const worker = await createWorker('eng', undefined, {
  langPath: process.env.TESSERACT_LANG_PATH
    || path.join(__dirname, '..', '..', '..', '..', 'vendor', 'tessdata'),
  cachePath: process.env.TESSERACT_CACHE_PATH || '/tmp/tesseract',
  gzip: false
});
```

Dockerfile — fetch the data at build time, not run time

```
RUN mkdir -p /app/vendor/tessdata \
 && curl -fsSL -o /app/vendor/tessdata/eng.traineddata \
      https://raw.githubusercontent.com/tesseract-ocr/tessdata/main/eng.traineddata

ENV TESSERACT_LANG_PATH=/app/vendor/tessdata
ENV TESSERACT_CACHE_PATH=/tmp/tesseract
```

Notes. node:20-slim has no curl by default — either add it in the same RUN layer with apt-get, or download the file in CI and COPY it in, which keeps the runtime image smaller. Use the .gz variant with gzip:true if image size matters.

Verify. Build the image, run it with outbound internet blocked, and OCR a scanned PDF through the document vault. It must return text rather than a network error.

T6. Shut down gracefully on SIGTERM

BLOCKER | Any container platform. Causes truncated uploads and failed requests on every deploy. | Effort: 30 minutes

File(s): server/server.js (around line 180)

Nothing listens for SIGTERM, which is precisely the signal every orchestrator sends before stopping a container. Node's default behaviour is to exit immediately, so in-flight requests are cut off mid-response and, worse, an upload being written to disk is truncated. Every routine deploy therefore has a window in which user work is destroyed.

server/server.js — replace the bare app.listen(...) call

```
const { pool } = require('./db/pool');   // add to the imports at the top

const server = app.listen(PORT, () => {
  logger.info('SLF Practice Management Document Vault API Running', {
    context: 'SERVER', port: PORT, url: `http://localhost:${PORT}`
  });
});

function shutdown(signal) {
  logger.info('Shutting down: refusing new connections, draining in-flight requests.',
    { context: 'SERVER', signal });
  server.close(async () => {
    try { await pool.end(); } catch (err) {
      logger.error('Error closing the database pool', { context: 'SERVER', error: err.message });
    }
    process.exit(0);
  });
  // Never hang forever if a connection refuses to drain.
  setTimeout(() => {
    logger.error('Forced exit: drain timeout elapsed.', { context: 'SERVER' });
    process.exit(1);
  }, Number(process.env.SHUTDOWN_TIMEOUT_MS || 15000)).unref();
}

for (const signal of ['SIGTERM', 'SIGINT']) {
  process.on(signal, () => shutdown(signal));
}
```

Notes. Keep the drain timeout below the orchestrator's own kill grace period (Kubernetes defaults to 30 seconds) so the app exits on its own terms first.

Verify. Start a large upload, send SIGTERM to the process mid-upload, and confirm the upload completes and the process then exits 0 rather than dying instantly.

High priority — fix before the firm depends on it

T7. Verify the database server's TLS certificate

HIGH | Not a blocker, but it weakens transport security for privileged data. | Effort: 30 minutes

File(s): server/db/pool.js (lines 36 and 54)

When PGSSL=true the pool is configured with rejectUnauthorized:false. The connection is encrypted but the server is never authenticated, so anything able to intercept the route to the database can present its own certificate and read or alter every query — which here means privileged client documents, messages and trust-account movements.

server/db/pool.js — build the TLS options properly

```
const fs = require('node:fs');

function buildSsl() {
  if (process.env.PGSSL !== 'true') return false;
  const ssl = {
    // Default to verifying. PGSSL_INSECURE exists only as a deliberate,
    // visible escape hatch for a self-signed dev database.
    rejectUnauthorized: process.env.PGSSL_INSECURE !== 'true'
  };
  if (process.env.PGSSLROOTCERT) {
    ssl.ca = fs.readFileSync(process.env.PGSSLROOTCERT, 'utf8');
  }
  return ssl;
}

// then use `ssl: buildSsl()` in BOTH branches of buildPoolConfig()
```

Notes. Managed providers publish a CA bundle — point PGSSLROOTCERT at it. While editing this file, also set pool sizing, which is currently left at node-pg's default of 10: add max, idleTimeoutMillis and connectionTimeoutMillis, and keep max x instance-count below the database's own connection limit.

Verify. Point PGSSLROOTCERT at a deliberately wrong CA and confirm the connection is refused.

T8. Run the container as a non-root user

HIGH | Not a blocker, but most platforms and security reviews will reject a root container. | Effort: 20 minutes

File(s): Dockerfile

No USER directive, so the application runs as root. Any remote-code-execution bug — and this app parses untrusted PDFs, Office documents and images — then executes with full privileges inside the container.

Dockerfile — add before CMD

```
# The node:20-slim image already ships an unprivileged `node` user.
RUN mkdir -p /var/lib/slf/uploads /tmp/tesseract \
 && chown -R node:node /var/lib/slf /tmp/tesseract /app
USER node
```

Notes. Any volume mounted at STORAGE_ROOT must be writable by uid 1000 (node). Also run the container with an init process (docker run --init, or Kubernetes' default) so signals reach Node properly alongside T6.

Verify. docker exec into the running container and confirm `whoami` returns node, then upload a file.

T9. Refuse to run the seed script in production

HIGH | Not a blocker, but one mistyped command overwrites live credentials. | Effort: 10 minutes

File(s): server/db/seed.js

The seed script has no environment guard. Run against production it inserts demo firm data and, where a password env var is set, overwrites real users' password hashes. There is nothing in the script to stop that.

server/db/seed.js — add at the very start of the main routine

```
if (process.env.NODE_ENV === 'production' && process.env.ALLOW_PROD_SEED !== 'true') {
  logger.error(
    'Refusing to seed: NODE_ENV=production. This script inserts demo data and can ' +
    'overwrite real users\' passwords. Set ALLOW_PROD_SEED=true only if you are ' +
    'certain this is a brand-new, empty production database.',
    { context: 'SEED' }
  );
  process.exit(1);
}
```

Verify. Run NODE_ENV=production npm run seed and confirm it exits non-zero without writing anything.

T10. Make CI actually run the integration tests

HIGH | Not a blocker, but it means nothing is genuinely gating a release. | Effort: 30 minutes

File(s): .github/workflows/ci.yml

The database-backed tests skip themselves when TEST_DATABASE_URL is unset, and CI never sets it — the workflow's own comment acknowledges this. So the overwhelming majority of the suite, including every access-control and trust-accounting test, does not run. A green tick currently certifies almost nothing.

.github/workflows/ci.yml — add a Postgres service and point the suite at it

```
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: postgres
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: slf_vault_test
        ports: ['5432:5432']
        options: >-
          --health-cmd pg_isready --health-interval 10s
          --health-timeout 5s --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm test
        env:
          TEST_DATABASE_URL: postgresql://postgres:postgres@localhost:5432/slf_vault_test
          JWT_SECRET: ci-only-secret-not-for-production-min-32-chars
```

Notes. Add a step that fails the build if the run reports any skipped suite, so this can never silently regress to a vacuous pass.

Verify. Push a branch and confirm the run reports roughly 400 passing tests rather than a handful.

Medium priority — tightening and operational readiness

T11. Move the error handler below the page routes

MEDIUM | Correctness of error responses only. | Effort: 5 minutes

File(s): server/server.js (lines 79–123)

The global error-handling middleware is mounted before the /portal route and the single-page catch-all. Express only routes an error to handlers registered after the failing handler, so an error thrown while serving either page bypasses the custom handler and falls through to Express's default, which in production returns a bare stack-free 500 with no request id and no log line.

server/server.js — reorder

```
// Move the whole `app.use((err, req, res, next) => { ... })` block so that it
// sits AFTER app.get('/portal', ...) and app.get('*', ...).
// Order must be: routes -> page routes -> error handler (last).
```

Verify. Force an error in the /portal handler and confirm the JSON error shape and a correlated log line.

T12. Validate required configuration at boot

MEDIUM | Not a blocker; reduces misconfiguration risk. | Effort: 45 minutes

File(s): server/config/validateProductionConfig.js (new); server/server.js

The app already fails fast on PostgreSQL, ClamAV and KMS, which is excellent. It does not check the remaining production-only settings, so a deployment with no STORAGE_ROOT, no TRUST_PROXY_HOPS behind a proxy, or a missing ALLOWED_ORIGINS for a separately hosted frontend starts happily and misbehaves subtly later.

server/config/validateProductionConfig.js — new file, called from start()

```
function validateProductionConfig() {
  if (process.env.NODE_ENV === 'development') return;
  const problems = [];

  if (!process.env.STORAGE_ROOT && process.env.STORAGE_PROVIDER !== 's3') {
    problems.push('STORAGE_ROOT is not set and STORAGE_PROVIDER is not s3 - uploaded ' +
                  'documents would be written to ephemeral container storage and lost on restart.');
  }
  if ((process.env.JWT_SECRET || '').length < 32) {
    problems.push('JWT_SECRET must be at least 32 characters.');
  }
  if (process.env.PGSSL === 'true' && process.env.PGSSL_INSECURE === 'true') {
    problems.push('PGSSL_INSECURE=true disables database certificate verification.');
  }
  if (problems.length) {
    throw new Error('Refusing to start:\n  - ' + problems.join('\n  - '));
  }
}
```

Notes. Call it in start() alongside the existing checkClamAvRequirement / checkMessageEncryptionRequirement calls.

Verify. Start with STORAGE_ROOT unset and NODE_ENV=production; the process must exit non-zero with a clear reason.

T13. Tighten the JSON body limit and tidy the environment reference

MEDIUM | Not a blocker. | Effort: 20 minutes

File(s): server/server.js (lines 61–62); .env.example

express.json is configured with a 50 MB limit, but file uploads go through multer as multipart rather than JSON, so nothing legitimately posts a JSON body anywhere near that size. It is a free memory-exhaustion surface. Separately, .env.example advertises SESSION_SECRET, which no code reads — a vestigial variable that invites someone to believe sessions are configured somewhere.

server/server.js

```
app.use(express.json({ limit: process.env.JSON_BODY_LIMIT || '1mb' }));
app.use(express.urlencoded({ extended: true, limit: process.env.JSON_BODY_LIMIT || '1mb' }));
```

.env.example

```
# Remove the unused SESSION_SECRET line entirely, and uncomment/represent the
# variables production actually requires as a single clearly-marked block
# (see the Environment Variables table in the accompanying report).
```

Notes. Confirm before merging that no endpoint posts a large JSON payload — the Copilot chat history is the only plausible candidate and is far below 1 MB.

Verify. Run the full test suite; exercise the Copilot chat and a document upload.

T14. Pin the Node version and tighten the image

MEDIUM | Not a blocker. | Effort: 15 minutes

File(s): package.json; Dockerfile; .dockerignore

package.json declares no engines field, so a non-container deployment may run an untested Node version. The image also copies the whole working tree, including the test suite and any local storage directory that happens to exist.

package.json

```
"engines": { "node": ">=20 <23" },
```

.dockerignore — add

```
storage
server/test
docs
*.pdf
```

Verify. Rebuild and confirm the image size drops and no storage/ or test files are present inside it.

T15. Write the operational runbook

MEDIUM | Not a blocker for the first deploy; required before the firm depends on it. | Effort: Half a day, including the drill

File(s): docs/RUNBOOK.md (new)

There is no documented backup, restore or incident procedure. For a system holding the firm's documents, client trust ledger and privileged correspondence, that is the difference between an incident and a catastrophe.

Minimum contents

```
1. Backups: pg_dump schedule, retention, and a PERIODICALLY TESTED restore.
   The document storage volume/bucket needs its own backup - the database
   alone is not a complete backup, because file bytes live outside it.
2. The one-off message-encryption migration, in order:
     npm run backfill:encrypt-messages
     # verify it reports 0 remaining plaintext rows, then, by hand:
     db/manual-migrations/0007_drop_plaintext_message_body.sql
3. Log shipping and retention (the app already emits structured JSON).
4. Alerting on /api/v1/health, which reports postgres, clamav and kms
   separately - alert per dependency, not just on the aggregate.
5. Secret rotation: JWT_SECRET (invalidates all sessions - expected),
   the KMS key (rotateKey() retires and re-mints; retired keys must be
   kept forever or old messages become unreadable), and database
   credentials.
6. Restore drill: document how to bring the system back from backups.
```

Notes. The pending manual migration in item 2 is already written and waiting in the repository. It is deliberately outside db/migrations/ because it is destructive and must be run by a human.

Verify. Perform a restore drill into a scratch environment and confirm documents download correctly.

T16. Consider moving the session token out of localStorage

MEDIUM | Not a blocker. Larger change — schedule deliberately. | Effort: 1–2 days

File(s): server/controllers/authController.js; server/middleware/auth.js; app.js; client-portal.js

The JWT is stored in localStorage, so any script that executes on the origin can read it and walk away with a valid twelve-hour session. The codebase knows this and compensates with a strict Content-Security-Policy that forbids inline script, which is a genuinely strong mitigation — this is a documented, deliberate trade-off rather than an oversight.

Direction, if adopted

```
Issue the token as a cookie with HttpOnly, Secure, SameSite=Strict,
then read it in authMiddleware from req.cookies instead of the
Authorization header. Because the API is same-origin with the frontend,
SameSite=Strict alone blocks cross-site request forgery; no CSRF token
is needed while that holds.

Touches every fetch() call in app.js and client-portal.js, plus the
login/logout flows and the public invite and signing pages.
```

Notes. Do not attempt this in the same change as the blockers above. It is a cross-cutting refactor with its own test surface, and the current CSP already removes the most common path to exploiting it.

Verify. Full suite, plus manual sign-in, sign-out, invite acceptance and document signing.

Environment variables required in production

Variable	Required	Purpose
DATABASE_URL	Yes	Full PostgreSQL connection string. Or PGHOST/PGUSER/PGPASSWORD/PGDATABASE/PGPORT.
JWT_SECRET	Yes	Token signing key, 32+ characters. Rotating it signs everyone out.
NODE_ENV	Yes	Must be production. Anything else relaxes the ClamAV and KMS requirements.
STORAGE_ROOT	Yes (T1)	Absolute path to the persistent uploads volume, e.g. /var/lib/slf/uploads.
CLAMAV_HOST / CLAMAV_PORT	Yes	Malware scanner. Boot fails without it outside development.
KMS_KEY_ARN	Yes	Master key for message encryption. Boot fails without it outside development.
TRUST_PROXY_HOPS	Yes (T3)	Number of proxies in front of the app. 1 behind a single load balancer.
PGSSL / PGSSLROOTCERT	Recommended	Enable and verify TLS to the database (T7).
ALLOWED_ORIGINS	If split frontend	Comma-separated origins. Omit entirely when the app serves its own UI.
AI_PROVIDER / AI_PROVIDER_API_KEY	Optional	Copilot, matter analysis and drafting. Without it those return 503; the rest of the app works.
TESSERACT_LANG_PATH	Yes (T5)	Path to the vendored eng.traineddata.
AUTH_RATE_LIMIT_MAX / _WINDOW_MS	Optional	Defaults to 10 attempts per 15 minutes.
SHUTDOWN_TIMEOUT_MS	Optional	Drain window on SIGTERM (T6). Default 15000.

A note on what is not in this list

The unconnected third-party vendors described in the system overview — notification delivery, the court-data feed and a commercial e-signature provider — are deliberately excluded here. They are commercial decisions rather than deployment defects: the system runs correctly without them and says honestly that they are not connected. None of them blocks going live.

Equally, nothing in this report asks for a rewrite. Every change listed is local, reversible and testable, and the largest single item outside the optional session-cookie refactor is a few hours of work.

Findings are based on a direct reading of server/server.js, server/db/pool.js, server/db/migrate.js, server/services/fileService.js, server/middleware/authRateLimit.js, server/middleware/securityHeaders.js, server/services/ocr/providers/tesseractProvider.js, the Dockerfile, .dockerignore, package.json and .github/workflows/ci.yml as they stand in the working tree. Line numbers refer to that same state.